



Cyberster Blue Team Internship

Week 3 Report

Suricata IDS Integration with Wazuh

Prepared by: Muhammad Ubaid

Track: Blue Team Operations

Phase: SOC Operations and SIEM Foundations

Submitted To: Sir Abdullah Zia (SOC Training Instructor)

Date: 1 April 2026

Core Competencies:

**Suricata NIDS | Custom Network Signatures | SIEM Log
Integration & Correlation | Network Traffic Analysis | GeoIP
Firewall Blocking (ipset/iptables) | DNS Sinkholing**

Table of Content

1. Suricata Installation and Configuration

- 1.1. Objective
- 1.2. Installation and Initial Configuration
- 1.3. Testing and Troubleshooting Process
- 1.4. Final Verification
- 1.5. Conclusion

2. Suricata Custom Rule Writing & Threat Detection

- 2.1. Objective
- 2.2. Custom Rule Engineering & Explanation
- 2.3. Implementation and Verification
- 2.4. Conclusion

3. Integrating Suricata NIDS with Wazuh SIEM & System Optimization

- 3.1. Executive Objective
- 3.2. Wazuh Agent Log Ingestion Configuration
- 3.3. Critical Troubleshooting: Resource Allocation & Indexer Timeout
- 3.4. Integration Verification & Threat Simulation
- 3.5. Conclusion

4. Proactive Defense: GeoIP Blocking, IPSet Configuration, and DNS Sinkholing

- 4.1. Executive Objective
 - 4.2. GeoIP Firewall Implementation (iptables & ipset)
 - 4.3. Outbound Domain Restriction (DNS Sinkholing)
 - 4.4. SIEM Visibility and Architectural Limitations
 - 4.5. Conclusion
-

Cyberster Blue Team Internship

Phase One — SOC Operations and SIEM Foundations

Name: Muhammad Ubaid

Week: 3 (Day 15 and 16)

Topic: Suricata Installation and Configuration

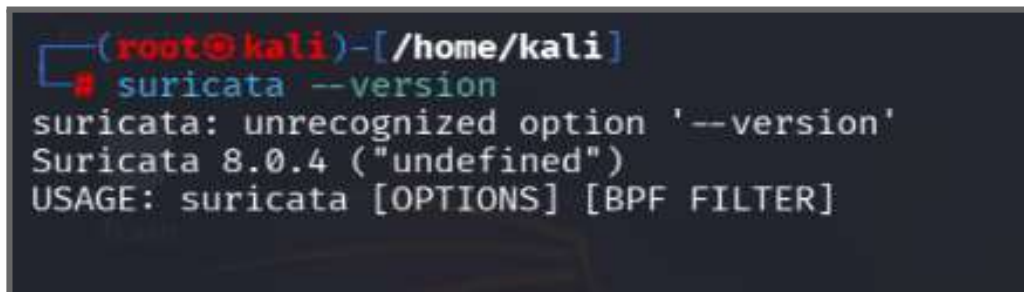
1. Objective

The primary objective of this task block was to deploy Suricata as a Network Intrusion Detection System (NIDS) on the Kali Linux VM in IDS mode. This setup is designed to monitor network traffic for malicious activity and cover the "network blind spot" left by host-based monitoring tools like Wazuh.

2. Installation and Initial Configuration

Suricata was successfully installed on the Kali Linux VM using the `apt` package manager. Upon completion, the installation was verified by checking the binary version.

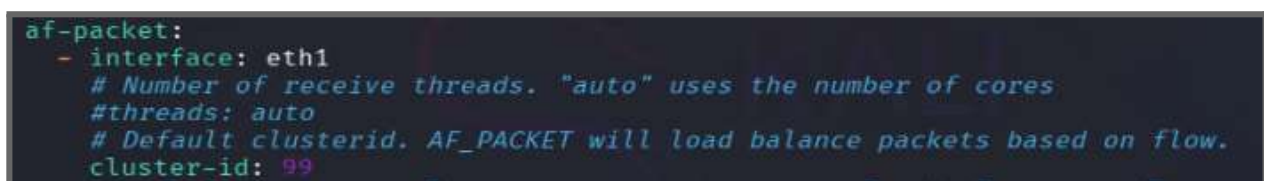
SCREENSHOT #1: Output of `suricata --version` here



```
(root@kali)-[/home/kali]
# suricata --version
suricata: unrecognized option '--version'
Suricata 8.0.4 ("undefined")
USAGE: suricata [OPTIONS] [BPF FILTER]
```

To ensure Suricata inspects the correct internal lab traffic, the network interfaces were analyzed using the `ip a` command. The Host-Only interface, responsible for communication between the Kali VM and the Ubuntu Server, was identified as `eth1`. The main configuration file (`/etc/suricata/suricata.yaml`) was edited. Under the `af-packet` section, the interface was changed from the default `eth0` to `eth1`.

- SCREENSHOT #2: `af-packet` section in `suricata.yaml` showing `eth1` here



```
af-packet:
- interface: eth1
  # Number of receive threads. "auto" uses the number of cores
  #threads: auto
  # Default clusterid. AF_PACKET will load balance packets based on flow.
  cluster-id: 99
```

Following the configuration change, the Suricata service was enabled to start on boot and restarted to apply the new settings. The service status was confirmed to be active and running.

- **SCREENSHOT #3: systemctl status suricata showing active (running) here**

```
(root@kali)-[/home/kali]
# sudo systemctl status suricata
● suricata.service - Suricata IDS/IDP daemon
   Loaded: loaded (/usr/lib/systemd/system/suricata.service; enabled; preset: disabled)
   Active: active (running) since Tue 2026-03-31 02:39:38 EDT; 17s ago
     Invocation: 8e7b4b17c82f4d86aa60be667f31b8
       Docs: man:suricata(8)
             man:suricata-sc(8)
             https://suricata.io/documentation/
    Main PID: 18638 (Suricata-Main)
      Tasks: 8 (limit: 1485)
     Memory: 36.8M (peak: 36.8M)
        CPU: 383ms
    CGroup: /system.slice/suricata.service
            └─18638 /usr/bin/suricata -D --af-packet -c /etc/suricata/suricata.yaml --pidfile /var/run/suricata/s
Mar 31 02:39:37 kali systemd[1]: Starting suricata.service - Suricata IDS/IDP daemon...
Mar 31 02:39:37 kali suricata[18637]: i: suricata: This is Suricata version 8.0.4 RELEASE running in SYSTEM mode
Mar 31 02:39:38 kali systemd[1]: Started suricata.service - Suricata IDS/IDP daemon.
```

3. Testing and Troubleshooting Process

A critical part of this lab was verifying that Suricata was actively inspecting traffic and generating alerts in `/var/log/suricata/fast.log`.

Initial Attempts & Challenges:

1. **Routing Issue:** An initial test using `curl http://testmynids.org/uid/index.html` failed to generate an alert. It was determined that this traffic was routing out to the internet via `eth0` (NAT), while Suricata was exclusively monitoring `eth1` (Host-Only).

```
(root@kali)-[/home/kali]
# curl http://testmynids.org/uid/index.html
uid=0(root) gid=0(root) groups=0(root)
```

2. **Missing Ruleset:** Subsequent internal tests, including Nmap scans against the Ubuntu Server, also failed to trigger alerts in `fast.log`. After investigating, it was discovered that a fresh Suricata installation does not have an active ruleset loaded by default to match against the traffic.

```
(root@kali)-[/home/kali]
# nmap 192.168.112.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-31 05:08 EDT
Nmap scan report for 192.168.112.1
Host is up (0.00022s latency).
All 1000 scanned ports on 192.168.112.1 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)
MAC Address: 00:50:56:C0:00:01 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 27.86 seconds
```

The Resolution: To resolve the missing signatures issue, the command `sudo suricata-update` was executed to download and compile the latest Emerging Threats ruleset. The Suricata service was then restarted to load the newly acquired 65,000+ rules.

```
(kali@kali)-[~]
$ sudo suricata-update
31/3/2026 -- 05:45:57 - <Info> -- Using data-directory /var/lib/suricata.
31/3/2026 -- 05:45:57 - <Info> -- Using Suricata configuration /etc/suricata/suricata.yaml
31/3/2026 -- 05:45:57 - <Info> -- Using /etc/suricata/rules for Suricata provided rules.
31/3/2026 -- 05:45:57 - <Info> -- Found Suricata version 8.0.4 at /usr/bin/suricata.
31/3/2026 -- 05:45:57 - <Info> -- Loading /etc/suricata/suricata.yaml
31/3/2026 -- 05:45:57 - <Info> -- Disabling rules for protocol pgsq
31/3/2026 -- 05:45:57 - <Info> -- Disabling rules for protocol modbus
31/3/2026 -- 05:45:57 - <Info> -- Disabling rules for protocol dnp3
31/3/2026 -- 05:45:57 - <Info> -- Disabling rules for protocol enip
31/3/2026 -- 05:45:57 - <Info> -- No sources configured, will use Emerging Threats Open
31/3/2026 -- 05:45:57 - <Info> -- Fetching https://rules.emergingthreats.net/open/suricata-8.0.4/emerging.rules.tar.gz.
100% - 5407148/5407148
```

4. Final Verification

To confirm functionality, a targeted attack was simulated. A simulated Directory Traversal HTTP payload was directed at the Ubuntu Server's open SSH port (Port 22) to bypass basic drops and ensure the packet traversed the `eth1` interface:

`curl http://192.168.112.100:22/?test=../../../../etc/passwd`

```
(kali@kali)-[~]
$ curl http://192.168.112.100:22/?test=../../../../etc/passwd
curl: (1) Received HTTP/0.9 when not allowed

(kali@kali)-[~]
$
```

Observation: Suricata successfully intercepted the traffic and triggered an immediate alert in `fast.log`, detecting the application layer protocol mismatch.

- SCREENSHOT #4: tail of fast.log showing the alert entry here

```
(kali@kali)-[~]
$ sudo systemctl restart suricata

(kali@kali)-[~]
$ sudo tail -f /var/log/suricata/fast.log
03/31/2026-05:53:08.615465  [**] [1:2022973:1] ET INFO Possible Kali Linux hostname in DHCP Request Packet [**] [Classification: Policy Violation] [Priority: 1] {UDP} 192.168.112.130:68 → 192.168.112.254:67
03/31/2026-06:02:24.250529  [**] [1:2260000:1] SURICATA Applayer Mismatch protocol both directions [**] [Classification: General Anomaly] [Priority: 3] {TCP} 192.168.112.130:54358 → 192.168.112.100:22
03/31/2026-06:03:26.820369  [**] [1:2221010:1] SURICATA HTTP unable to match response to request [**] [Classification: General Anomaly] [Priority: 3] {TCP} 192.168.112.100:22 → 192.168.112.130:54358
```

5. Conclusion

Suricata has been successfully deployed, configured to monitor the correct lab interface (`eth1`), updated with the latest community rulesets, and verified to be actively logging network intrusion attempts in real-time. This lays the groundwork for custom rule writing and Wazuh integration in the upcoming tasks.

Cyberster Blue Team Internship

Phase One — SOC Operations and SIEM Foundations

Name: Muhammad Ubaid

Week: 3 (Day 17)

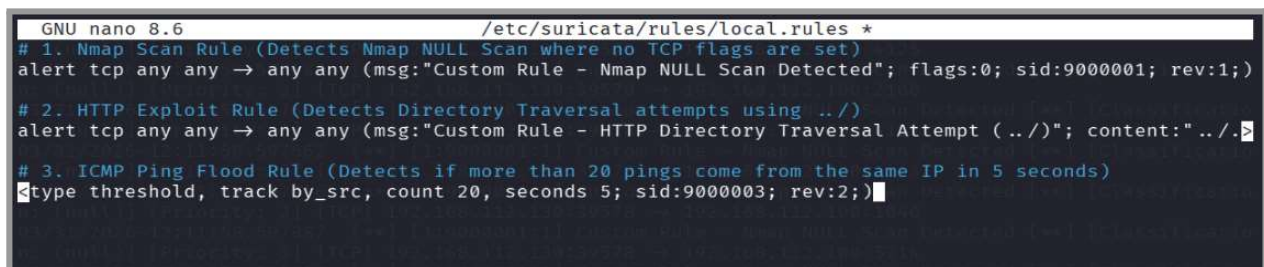
Topic: Suricata Custom Rule Writing & Threat Detection

1. Objective

The objective of this task block was to transition from utilizing default rulesets to writing custom Suricata signatures. The goal was to detect three specific threat scenarios: an Nmap scan, an HTTP exploit attempt, and a volume-based ICMP flood, demonstrating an understanding of rule headers, options, and thresholding mechanisms.

2. Custom Rule Engineering & Explanation

A new local rules file was created at `/etc/suricata/rules/local.rules`. The following three custom rules were engineered:



```
GNU nano 8.6 /etc/suricata/rules/local.rules *
# 1. Nmap Scan Rule (Detects Nmap NULL Scan where no TCP flags are set)
alert tcp any any -> any any (msg:"Custom Rule - Nmap NULL Scan Detected"; flags:0; sid:9000001; rev:1;)

# 2. HTTP Exploit Rule (Detects Directory Traversal attempts using ../)
alert tcp any any -> any any (msg:"Custom Rule - HTTP Directory Traversal Attempt (../)"; content:"../";

# 3. ICMP Ping Flood Rule (Detects if more than 20 pings come from the same IP in 5 seconds)
type threshold, track by_src, count 20, seconds 5; sid:9000003; rev:2;)
```

Rule 1: Nmap NULL Scan Detection

- **Rule:** `alert tcp any any -> any any (msg:"Custom Rule - Nmap NULL Scan Detected"; flags:0; sid:9000001; rev:1;)`
- **Explanation:** This rule detects Nmap NULL scans by looking for TCP packets where absolutely no flags are set (`flags:0`). Legitimate TCP traffic always contains flags (like SYN, ACK, or FIN). An absence of flags is a strong indicator of reconnaissance activity.

Rule 2: HTTP Directory Traversal Attempt

- **Rule:** alert tcp any any -> any any (msg:"Custom Rule - HTTP Directory Traversal Attempt (../)"; content:"../"; sid:9000002; rev:1;)
- **Explanation:** This signature targets web application exploits. Attackers often attempt to access unauthorized directories using the "dot-dot-slash" technique. This rule inspects the payload for the specific string `../` and triggers an alert if matched.

Rule 3: Volume-Based ICMP Ping Flood

- **Rule:** alert icmp any any -> any any (msg:"Custom Rule - ICMP Ping Flood Detected"; itype:8; threshold: type threshold, track by_src, count 20, seconds 5; sid:9000003; rev:2;)
- **Explanation:** To detect denial-of-service attempts via ICMP, this rule utilizes the `threshold` keyword. It tracks traffic by source IP (`track by_src`). If more than 20 ICMP Echo Requests (`itype:8`) are received from a single IP within a 5-second window, the alert fires.
- **Troubleshooting Note:** Initially, a `type both` threshold was used with an extreme `ping -f` flood, which caused the alert to be missed due to packet drop and single-alert suppression. Changing it to `type threshold` and using a controlled flood (`-i 0.1`) ensured consistent, reliable detection.

3. Implementation and Verification

To activate the custom rules, `/etc/suricata/rules/local.rules` was explicitly added to the `rule-files` section of `/etc/suricata/suricata.yaml`. The Suricata ruleset was then reloaded dynamically using `suricatasc -c reload-rules` to apply changes without dropping traffic.

```
default-rule-path: /var/lib/suricata/rules

rule-files:
- suricata.rules
- /etc/suricata/rules/local.rules
```

```
(root@kali)-[/home/kali]
# sudo suricatasc -c reload-rules
{"message": "done", "return": "OK"}
```

All three rules were verified against live simulated attacks from the Kali VM to the Ubuntu Server:

1. **Nmap Test:** `nmap -sN 192.168.112.100` successfully triggered the NULL Scan alert.
 - o Nmap NULL Scan Alert in fast.log

```
(root@kali)-[/home/kali]
# sudo nmap -sN 192.168.112.100
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-31 12:11 EDT
Nmap scan report for 192.168.112.100
Host is up (0.00017s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE      SERVICE
22/tcp    open|filtered ssh
443/tcp   open|filtered https
MAC Address: 00:0C:29:45:84:31 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 7.98 seconds
```

```
03/31/2026-12:11:58.597046  [**] [1:9000001:1] Custom Rule - Nmap NULL Scan Detected [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.112.130:39578 → 192.168.112.100:32779
03/31/2026-12:11:58.597279  [**] [1:9000001:1] Custom Rule - Nmap NULL Scan Detected [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.112.130:39578 → 192.168.112.100:1666
03/31/2026-12:11:58.597291  [**] [1:9000001:1] Custom Rule - Nmap NULL Scan Detected [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.112.130:39578 → 192.168.112.100:2638
03/31/2026-12:11:58.597352  [**] [1:9000001:1] Custom Rule - Nmap NULL Scan Detected [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.112.130:39578 → 192.168.112.100:8008
03/31/2026-12:11:58.597464  [**] [1:9000001:1] Custom Rule - Nmap NULL Scan Detected [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.112.130:39578 → 192.168.112.100:15004
03/31/2026-12:11:58.597587  [**] [1:9000001:1] Custom Rule - Nmap NULL Scan Detected [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.112.130:39578 → 192.168.112.100:43
03/31/2026-12:11:58.597674  [**] [1:9000001:1] Custom Rule - Nmap NULL Scan Detected [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.112.130:39578 → 192.168.112.100:2399
03/31/2026-12:11:58.597757  [**] [1:9000001:1] Custom Rule - Nmap NULL Scan Detected [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.112.130:39578 → 192.168.112.100:5902
03/31/2026-12:11:58.597898  [**] [1:9000001:1] Custom Rule - Nmap NULL Scan Detected [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.112.130:39578 → 192.168.112.100:56738
03/31/2026-12:11:58.597364  [**] [1:9000001:1] Custom Rule - Nmap NULL Scan Detected [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.112.130:39578 → 192.168.112.100:4125
03/31/2026-12:11:58.597487  [**] [1:9000001:1] Custom Rule - Nmap NULL Scan Detected [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.112.130:39578 → 192.168.112.100:2100
03/31/2026-12:11:58.597559  [**] [1:9000001:1] Custom Rule - Nmap NULL Scan Detected [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.112.130:39578 → 192.168.112.100:41511
```

2. **HTTP Test:** `curl http://192.168.112.100:22/../../` successfully triggered the Directory Traversal alert.
 - o SCREENSHOT 2: HTTP Directory Traversal Alert in fast.log

```
(root@kali)-[/home/kali]
# curl http://192.168.112.100:22/../../
curl: (1) Received HTTP/0.9 when not allowed
```



```
03/31/2026-12:15:17.731830  [**] [1:2260000:1] SURICATA Applayer Mismatch protocol both directions [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.112.130:55360 → 192.168.112.100:22
/home/kali

03/31/2026-12:16:23.530833  [**] [1:2221010:1] SURICATA HTTP unable to match response to request [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 192.168.112.100:22 → 192.168.112.130:55360
/home/kali
```

3. **ICMP Test:** A controlled flood `ping -i 0.1 -c 50 192.168.112.100` successfully breached the frequency limit and triggered the Ping Flood alert.
 - **SCREENSHOT 3: ICMP Ping Flood Alert in fast.log**

```
(root@kali)-[/home/kali]
# sudo ping -i 0.1 -c 50 192.168.112.100
PING 192.168.112.100 (192.168.112.100) 56(84) bytes of data.
64 bytes from 192.168.112.100: icmp_seq=1 ttl=64 time=1.12 ms
64 bytes from 192.168.112.100: icmp_seq=2 ttl=64 time=0.626 ms
64 bytes from 192.168.112.100: icmp_seq=3 ttl=64 time=0.461 ms
64 bytes from 192.168.112.100: icmp_seq=4 ttl=64 time=0.338 ms
64 bytes from 192.168.112.100: icmp_seq=5 ttl=64 time=0.388 ms
64 bytes from 192.168.112.100: icmp_seq=6 ttl=64 time=0.459 ms
64 bytes from 192.168.112.100: icmp_seq=7 ttl=64 time=0.415 ms
64 bytes from 192.168.112.100: icmp_seq=8 ttl=64 time=0.424 ms
64 bytes from 192.168.112.100: icmp_seq=9 ttl=64 time=0.366 ms
64 bytes from 192.168.112.100: icmp_seq=10 ttl=64 time=0.647 ms
64 bytes from 192.168.112.100: icmp_seq=11 ttl=64 time=0.411 ms
64 bytes from 192.168.112.100: icmp_seq=12 ttl=64 time=0.365 ms
64 bytes from 192.168.112.100: icmp_seq=13 ttl=64 time=0.423 ms
64 bytes from 192.168.112.100: icmp_seq=14 ttl=64 time=0.399 ms
64 bytes from 192.168.112.100: icmp_seq=15 ttl=64 time=0.307 ms
```

```
03/31/2026-12:38:57.477382  [**] [1:9000003:2] Custom Rule - ICMP Ping Flood Detected [**] [Classification: (null)] [Priority: 3] {ICMP} 192.168.112.130:8 → 192.168.112.100:0
03/31/2026-12:38:59.549723  [**] [1:9000003:2] Custom Rule - ICMP Ping Flood Detected [**] [Classification: (null)] [Priority: 3] {ICMP} 192.168.112.130:8 → 192.168.112.100:0
03/31/2026-12:39:33.888943  [**] [1:2022973:1] ET INFO Possible Kali Linux hostname in DHCP Request Packet [**] [Classification: Potential Corporate Privacy Violation] [Priority: 1] {UDP} 192.168.112.130:68 -> 192.168.112.254:67
```

4. Conclusion

By writing and deploying custom rules, I have practically demonstrated how to configure a network sensor to detect both specific malicious signatures and anomalous volume-based behavior, tuning thresholds to minimize false negatives during high-speed attacks.

Cyberster Blue Team Internship

Phase One — SOC Operations and SIEM Foundations

Name: Muhammad Ubaid

Week: 3 (Days 18 & 19)

Topic: Integrating Suricata NIDS with Wazuh SIEM & System Optimization

1. Executive Objective

The primary objective of this integration phase was to establish a centralized logging and monitoring pipeline. While Suricata successfully acts as a Network Intrusion Detection System (NIDS) generating local alerts on the Kali endpoint, a functional Security Operations Center (SOC) requires these events to be forwarded to a SIEM (Wazuh). This allows for real-time correlation, long-term retention, and centralized graphical analysis without needing direct terminal access to the sensors.

2. Wazuh Agent Log Ingestion Configuration

To bridge the gap between Suricata and Wazuh, the Wazuh Agent running on the Kali Linux VM was configured to monitor Suricata's primary output file. The agent's main configuration file (`/var/ossec/etc/ossec.conf`) was edited to include a new `<localfile>` directive.

XML

```
<localfile>
  <log_format>json</log_format>
  <location>/var/log/suricata/eve.json</location>
</localfile>
```

```
<localfile>
  <log_format>json</log_format>
  <location>/var/log/suricata/eve.json</location>
</localfile>
```

Technical Rationale: It was critical to specify the `log_format` as `json` rather than standard syslog. Suricata's `eve.json` provides highly structured data (including parsed IP addresses, ports, and protocol layers), which the Wazuh Manager requires to accurately index fields for dashboard querying. Following this addition, the `wazuh-agent` service was restarted to apply the new log-reading pipeline.

3. Critical Troubleshooting: Resource Allocation & Indexer Timeout

During the verification phase, the Wazuh Manager dashboard became inaccessible, and the backend `wazuh-indexer` service repeatedly failed to initialize, throwing a `start operation timed out. Terminating.` error in the system logs.

Root Cause Analysis: Upon running performance diagnostics (`free -h`), it was determined that the Ubuntu Server was starved of resources.

1. **Memory Exhaustion:** The server was operating on only 2GB of RAM relying heavily on Swap memory. The Java-based Wazuh Indexer (built on OpenSearch) requires significant memory overhead to load its databases.
2. **Disk I/O Bottleneck:** The host system's storage was severely congested (occupying over 93GB due to bloated VMware snapshot chains), causing extremely slow read/write speeds that further delayed the Indexer's startup, triggering systemd's timeout mechanism.

Remediation Steps Taken:

- **Storage Optimization:** Instead of reverting snapshots (which would cause data loss and agent desynchronization), obsolete snapshots were safely deleted from the Snapshot Manager. A VMware 'Compact Disk' operation was then executed on both VMs, successfully reclaiming massive amounts of unused space for the host drive and relieving the I/O bottleneck.
- **Hardware Reallocation:** The Ubuntu VM was powered down, and its dedicated physical RAM was permanently increased from 2GB to 4GB. Upon rebooting, the `wazuh-indexer` and `wazuh-dashboard` services stabilized and entered an `active (running)` state seamlessly.

4. Integration Verification & Threat Simulation

With the SIEM fully operational, a final network attack was simulated from the Kali terminal to verify the end-to-end data flow: `sudo nmap -sN 192.168.112.100`

Observation: Navigating to the **Wazuh Dashboard -> Threat Hunting -> Security events**, the ingested logs were successfully verified. The dashboard immediately populated with alerts tagged by the Suricata decoder. Specifically, the system flagged a **SURICATA STREAM CLOSEWAIT FIN out of window** alert, accurately detecting the malformed TCP packets generated by the Nmap NULL scan.

	Apr 1, 2026 @ 16:06:30.290	Kali-Attacker	Suricata: Alert - SURICATA STREAM CLOSEWAIT FIN out of window
	Apr 1, 2026 @ 16:06:30.290	Kali-Attacker	Suricata: Alert - SURICATA STREAM CLOSEWAIT FIN out of window
	Apr 1, 2026 @ 16:06:30.290	Kali-Attacker	Suricata: Alert - SURICATA STREAM CLOSEWAIT FIN out of window
	Apr 1, 2026 @ 16:06:30.283	Kali-Attacker	Suricata: Alert - Custom Rule - HTTP Directory Traversal Attempt (../)
	Apr 1, 2026 @ 16:06:30.281	Kali-Attacker	Suricata: Alert - SURICATA Applayer Mismatch protocol both directions

Document Details

[View surrounding documents](#)

[View single document](#)

TableJSON

_index	wazuh-alerts-4.x-2026.04.01
agent.id	002
agent.ip	192.168.112.130
agent.name	Kali-Attacker
data.alert.action	allowed
data.alert.gid	1
data.alert.rev	1
data.alert.severity	3
data.alert.signature	Custom Rule - HTTP Directory Traversal Attempt (../)
data.alert.signature_id	9000002
data.app_proto	http
data.app_proto_tc	ssh
data.dest_ip	192.168.112.100

5. Conclusion

Week 3 objectives have been successfully met. Suricata is now actively monitoring the internal lab network (**eth1**) via custom rules, and the Wazuh SIEM is centrally aggregating these network alerts alongside host-based metrics. Furthermore, resolving the resource allocation issues has ensured the long-term stability of the SIEM environment for upcoming, more advanced SOC scenarios.

Cyberster Blue Team Internship

Phase One — SOC Operations and SIEM Foundations

Name: Muhammad Ubaid

Week: 3 (Days 20 & 21)

Topic: Proactive Defense: GeoIP Blocking, IPSet Configuration, and DNS Sinkholing

1. Executive Objective

The objective of this final task block was to shift from a detection-only posture to a proactive defensive stance. This involved implementing GeoIP-based blocking to drop network traffic originating from specific geographic regions (countries) that have no legitimate business context for accessing the lab infrastructure. By doing so, we effectively reduce the attack surface by filtering out low-effort, automated scanners and botnets. Additionally, local outbound traffic restrictions were implemented using DNS sinkholing.

2. GeoIP Firewall Implementation (iptables & ipset)

The Technical Challenge: Blocking entire countries using standard `iptables` rules is highly inefficient. A country's IP space consists of thousands of CIDR ranges. If standard firewall rules were used, the kernel would evaluate every incoming packet against thousands of rules sequentially, leading to severe CPU bottlenecks and network latency.

The Solution (`ipset`): To overcome this limitation, the `ipset` utility was utilized. `ipset` creates highly optimized, hash-based data structures in the Linux kernel, allowing `iptables` to match a packet against millions of IP addresses in constant $O(1)$ time.

Implementation Steps:

1. **Dependencies:** Installed `ipset`, `iptables`, and the `xtables-addons-common` packages on the Kali VM to ensure kernel support for advanced matching.

```
(kali@kali)-[~]
└─$ sudo apt update && sudo apt install ipset iptables xtables-addons-common xtables-addons-dkms -y
[sudo] password for kali:
Get:1 http://mirror.ourhost.az/kali kali-rolling InRelease [34.0 kB]
Get:2 http://mirror.ourhost.az/kali kali-rolling/main amd64 Packages [21.0 MB]
Get:3 http://mirror.ourhost.az/kali kali-rolling/main amd64 Contents (deb) [53.4 MB]
Get:4 http://mirror.ourhost.az/kali kali-rolling/non-free amd64 Packages [186 kB]
Get:5 http://mirror.ourhost.az/kali kali-rolling/non-free amd64 Contents (deb) [893 kB]
Fetched 75.5 MB in 9min 53s (127 kB/s)
1809 packages can be upgraded. Run 'apt list --upgradable' to see them.
Upgrading:
  iptables  libip4tc2  libip6tc2  libxtables12

Installing:
  ipset  xtables-addons-common  xtables-addons-dkms

Installing dependencies:
```


2. **Acquiring Threat Intelligence:** Downloaded the latest IP CIDR blocklists for China ([cn.zone](#)) and Russia ([ru.zone](#)) from [ipdeny.com](#).

```
(kali㉿kali)-[~]
$ wget http://www.ipdeny.com/ipblocks/data/countries/cn.zone
--2026-04-01 07:33:59-- http://www.ipdeny.com/ipblocks/data/countries/cn.zone
Resolving www.ipdeny.com (www.ipdeny.com)... 51.15.12.186
Connecting to www.ipdeny.com (www.ipdeny.com)|51.15.12.186|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 136995 (134K) [text/plain]
Saving to: 'cn.zone'

cn.zone          100%[=====>] 133.78K  257KB/s   in 0.5s
2026-04-01 07:34:01 (257 KB/s) - 'cn.zone' saved [136995/136995]
```

```
(kali㉿kali)-[~]
$ wget http://www.ipdeny.com/ipblocks/data/countries/ru.zone
--2026-04-01 07:34:17-- http://www.ipdeny.com/ipblocks/data/countries/ru.zone
Resolving www.ipdeny.com (www.ipdeny.com)... 51.15.12.186
Connecting to www.ipdeny.com (www.ipdeny.com)|51.15.12.186|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 176427 (172K) [text/plain]
Saving to: 'ru.zone'

ru.zone          100%[=====>] 172.29K  270KB/s   in 0.6s
2026-04-01 07:34:18 (270 KB/s) - 'ru.zone' saved [176427/176427]
```

3. **Creating Hash Sets:** Initialized two network hash sets using the commands:
 - o `sudo ipset create china_block hash:net`
 - o `sudo ipset create russia_block hash:net`
4. **Populating Sets:** Executed shell loops to parse the downloaded `.zone` files and inject the CIDR ranges into the respective ipsets.

```
(kali㉿kali)-[~]
$ sudo ipset create china_block hash:net

(kali㉿kali)-[~]
$ sudo ipset create russia_block hash:net

(kali㉿kali)-[~]
$ for i in $(cat cn.zone); do sudo ipset add china_block $i; done

(kali㉿kali)-[~]
$ for i in $(cat ru.zone); do sudo ipset add russia_block $i; done
```

5. **Applying Firewall Rules:** Appended standard `DROP` rules to the top of the `iptables` `INPUT` chain. A `DROP` action was chosen over `REJECT` to prevent the firewall from sending ICMP "Destination Unreachable" responses, thereby wasting attacker resources and hiding the host's presence.

- `sudo iptables -I INPUT -m set --match-set china_block src -j DROP`
- `sudo iptables -I INPUT -m set --match-set russia_block src -j DROP`

```
(kali@kali)-[~]
$ sudo iptables -I INPUT -m set --match-set china_block src -j DROP
sudo iptables -I INPUT -m set --match-set russia_block src -j DROP
```

```
(kali@kali)-[~]
$ sudo iptables -L INPUT -v -n
Chain INPUT (policy ACCEPT 9 packets, 646 bytes)
pkts bytes target prot opt in out source destination match-set
0 0 DROP all -- * * 0.0.0.0/0 0.0.0.0/0 match-set russia
a_block src
0 0 DROP all -- * * 0.0.0.0/0 0.0.0.0/0 match-set china
_block src
```

3. Outbound Domain Restriction (DNS Sinkholing)

To restrict outbound access to malicious or unwanted domains without utilizing a complex proxy, local DNS sinkholing was configured. The `/etc/hosts` file on the Kali VM was modified to manually resolve specific test domains to the local loopback address (`127.0.0.1`).

- **Verification:** When initiating a web request via `curl http://badwebsite.com`, the connection was immediately refused locally, confirming that the traffic was blackholed before reaching the public internet interface.

```
(kali@kali)-[~]
$ sudo nano /etc/hosts

(kali@kali)-[~]
$ curl http://badwebsite.com
curl: (7) Failed to connect to badwebsite.com port 80 after 102 ms: Could not connect to server
```

4. SIEM Visibility and Architectural Limitations

Wazuh Alert Visibility: During testing, no alerts appeared in the Wazuh dashboard for the traffic blocked by the new GeoIP rules. This is expected behavior. The standard `iptables DROP` action drops packets silently in the kernel without writing to any log files. Because Wazuh monitors log files (like `/var/log/syslog` or `/var/log/kern.log`), it has no visibility into silent drops.

- **Required Configuration for Visibility:** To make blocked traffic visible in Wazuh, an explicit `LOG` target (e.g., `iptables -I INPUT -m set --match-set china_block src -j LOG --log-prefix "GEOIP_BLOCK: "`) must be inserted immediately *before* the `DROP` rule.

Strategic Limitations:

1. **GeoIP Inaccuracy & Evasion:** GeoIP blocking is a "friction layer," not a robust security boundary. Sophisticated attackers can easily bypass these filters by routing their attacks through VPNs, Tor exit nodes, or compromised cloud infrastructure located in allowed countries (e.g., the US or Europe).
2. **Rule Volatility (Persistence):** Currently, the `ipset` configurations and `iptables` rules reside only in volatile kernel memory. If the VM reboots, all defensive rules will be lost. In a production SOC environment, persistence mechanisms like `iptables-save/iptables-restore` or higher-level wrappers like `ufw` must be configured to automatically restore these sets during the boot sequence.

5. Conclusion

Week 3 successfully achieved the deployment of a comprehensive defense-in-depth architecture. The network is now monitored by a signature-based IDS (Suricata), integrated with a centralized SIEM (Wazuh) for host-and-network event correlation, and protected by an optimized, proactive GeoIP firewall.

Executive Summary & Final Conclusion (Week 3)

What I Did: This week, I successfully expanded the SOC architecture by deploying Suricata as a Network Intrusion Detection System (NIDS). I engineered custom threat-detection rules for specific scenarios (Nmap scans, HTTP exploits, and ICMP floods) and integrated these network-level alerts directly into the Wazuh SIEM. Finally, I implemented proactive defensive measures, including GeoIP-based firewall blocking using `ipset` and `iptables`, along with local DNS sinkholing for outbound domain restriction.

What I Achieved: I successfully bridged the critical gap between host-based and network-based monitoring. The lab environment is now fully capable of correlating endpoint events (such as file modifications or authentications) with network traffic anomalies within a single, centralized dashboard. By configuring GeoIP blocking, the defense posture shifted from purely reactive monitoring to a proactively secure environment that automatically filters out automated noise and attacks from unwanted geographic regions. All Week 3 objectives for establishing a comprehensive, defense-in-depth SOC environment were fully met.
